

Informe de auditoría técnica — SNIPER RSI v2

Archivo auditado: sniper_rsi_v2_fdfb.py

Fecha: 2026-07-20

Alcance: funcionamiento, operabilidad, seguridad, concurrencia, integración API/DB/Telegram

Excluido: decisiones de estrategia (niveles RSI, TP %, % capital, grid, trailing, criterios del auditor)

1. Resumen ejecutivo

Severidad	Cantidad	Impacto
-----	-----	-----
Crítica	4	Impide arranque o puede causar pérdidas / órdenes incorrectas
Alta	9	Fallos intermitentes o comportamiento incorrecto en producción
Media	12	Degradación, mantenimiento difícil, riesgo acumulado
Baja	8	Mejoras de robustez y observabilidad

Veredicto: el código compila y la arquitectura general es coherente, pero **no está listo para producción sin correcciones**. El bloqueador de arranque principal es la dependencia del módulo `auto_auditor`. En runtime, los bugs más graves son la contabilización multi-escalón por símbolo, la inconsistencia del cálculo RSI y el apalancamiento fijo en pares con tope distinto.

2. Bloqueantes de arranque

AUD-S01 · Import del módulo auditoría (CRÍTICA)

Ubicación: línea 31

```
import auto_auditor
```

Problema: el archivo subido se llama `auto_auditor_5053.py`. Python no lo encuentra → `ModuleNotFoundError` al iniciar.

Corrección: renombrar a `auto_auditor.py` en el mismo directorio que el bot, o cambiar el import y empaquetar como módulo.

AUD-S02 · Dependencias no declaradas (ALTA)

Paquetes requeridos: requests, numpy, pandas (stdlib: sqlite3, threading, hmac, etc.).

Problema: no hay `requirements.txt` ni verificación al arranque.

Corrección: agregar `requirements.txt` y chequeo opcional en `main()` con mensaje claro.

AUD-S03 · Ruta de base de datos relativa (MEDIA)

Ubicación: línea 40 — `DB_PATH = "sniper_rsi_v2.db"`

Problema: si el proceso se lanza desde otro directorio (systemd, cron, IDE), crea/usa otra DB vacía → pérdida de estado, órdenes huérfanas, trades duplicados.

Corrección: path absoluto basado en `__file__` o variable de entorno `SNIPER_DB_PATH`.

3. Seguridad y secretos

AUD-S04 · Credenciales en código fuente (CRÍTICA — seguridad)

Ubicación: líneas 34–37

API Key, API Secret y token/chat de Telegram en texto plano.

Riesgo: filtración del repo, backup, log o captura de pantalla → acceso total a la cuenta de futuros y control remoto del bot.

Corrección inmediata:

1. Rotar keys en Binance y BotFather.
 2. Cargar desde variables de entorno (BINANCE_API_KEY, etc.).
 3. Nunca commitear secretos.
-

AUD-S05 · Telegram sin filtro de chat_id (CRÍTICA — operación)

Ubicación: escuchar_telegram() líneas 913–980

Problema: cualquier usuario que escriba al bot puede ejecutar /cerrar, /pausa, /aplicar_lista, /auditar.

Corrección:

```
chat_id = str(upd.get("message", {}).get("chat", {}).get("id", ""))
if chat_id != str(TELEGRAM_CHAT):
    continue
```

AUD-S06 · parse_mode HTML sin escape (BAJA)

Problema: mensajes con datos de mercado raramente rompen HTML de Telegram; bajo riesgo pero mejorable con escape de <, >, & en símbolos dinámicos.

4. Integración Binance Futures

AUD-S07 · Apalancamiento fijo 25x vs máximo del símbolo (CRÍTICA)

Ubicación: CONFIG (apal: 25), fijar_leverage(), correr_auditoria() (apal hardcodeado 25)

Problema: pares como BABAUSDT (acciones tokenizadas) tienen máximo 10x en Binance Futures. POST /fapi/v1/leverage con 25 falla → sin órdenes en ese par.

Corrección: leer exchangeInfo → leverageBracket o probar leverage y usar min(apal_config, max_permitido).

AUD-S08 · Sin recvWindow ni sincronización de reloj (ALTA)

Problema: todas las firmas usan solo timestamp. Desfase del VPS > ~1s → errores -1021 Timestamp for this request is outside of the recvWindow.

Corrección: recvWindow=5000 y/o sync NTP; reintento con offset si falla.

AUD-S09 · Respuestas API no validadas sistemáticamente (ALTA)

Ejemplos:

- set_isolated(): marca margin_type_cache[symbol] = True sin leer status/body (líneas 459–471).
- fijar_leverage(): solo cachea si status_code == 200; errores silenciosos.
- cancelar_orden() / cancelar_todas(): traga excepciones sin log.

Impacto: el bot cree que el margen está aislado o el leverage fijado cuando no lo está.

Corrección: parsear JSON de error Binance, loggear código y mensaje, no cachear éxito sin confirmación.

AUD-S10 · Modo hedge no soportado (ALTA)

Problema: órdenes sin `positionSide`. En cuentas en **hedge mode** las órdenes fallan o abren el lado incorrecto.

Corrección: al arranque consultar `GET /fapi/v1/positionSide/dual`; abortar o forzar one-way con mensaje claro.

AUD-S11 · Filtro `MIN_NOTIONAL` no leído de `exchangeInfo` (ALTA)

Ubicación: `MIN_NOTIONAL = 5.0` fijo; `ajustar_qty()` solo usa `LOT_SIZE`.

Problema: Binance rechaza órdenes con notional real < mínimo del símbolo aunque qty sea válida.

Corrección: validar `quantity * price >= minNotional` del filtro correspondiente.

AUD-S12 · Sin manejo de `rate limit` -1003 (MEDIA)

Problema: ~17 símbolos cada 30s + monitor cada 5s + `openOrders` cada 5 min → riesgo de ban temporal.

Corrección: backoff exponencial, cola centralizada de requests, respetar headers `X-MBX-USED-WEIGHT - *`.

AUD-S13 · Formato de `quantity/price` en `query string` (MEDIA)

Problema: floats pasados directo a la URL; en activos muy baratos podría generarse notación científica.

Corrección: formatear con precisión de `stepSize/tickSize` como string decimal.

AUD-S14 · `STOP_MARKET` sin `workingType` explícito (BAJA)

Problema: en condiciones extremas, trigger por mark vs last price puede diferir de lo esperado.

Corrección: documentar o fijar `workingType=CONTRACT_PRICE` (o `MARK_PRICE`) según preferencia operativa.

5. Lógica de trading y bugs funcionales

AUD-S15 · RSI inconsistente entre decisión y precio límite (ALTA)

Función	Método RSI
-----	-----
<code>calcular_rsi_actual()</code>	EWM pandas (<code>ewm(alpha=1/14)</code>)
<code>precio_para_rsi()</code>	Wilder clásico (SMA inicial + suavizado)
<code>auto_auditor.rsi_wilder()</code>	Wilder clásico

Impacto: el lado BUY/SELL se decide con un RSI; el precio de la limit se calcula con otro. Desfase sistemático.

Corrección: una sola implementación Wilder compartida (módulo `rsi_utils.py`).

AUD-S16 · Ventana de velas inconsistente (MEDIA)

- `refrescar_grid`: `closes = df["close"].iloc[:-1]` (excluye vela en curso).
- `calcular_rsi_actual`: incluye vela en curso (`iloc[-1]`).

Impacto: refuerza el desfase señal vs precio objetivo.

Corrección: documentar y unificar criterio (recomendado: Wilder sobre velas cerradas para límites; RSI "live" solo para side con la misma serie extendida).

AUD-S17 · Multi-escalón mismo símbolo: `qty` mal repartida (CRÍTICA)

Ubicación: `monitor_tp_sl()` líneas 816–890

Problema: `posiciones_abiertas()` devuelve **posición agregada** por símbolo. Cada trade en DB tiene su qty, pero al cerrar:

```
disponible = abs(float(pos[symbol]))
q = ajustar_qty(symbol, min(qty, disponible))
```

Varios trades compiten por el mismo disponible sin reservar por escalón.

Escenario: 4 escalones llenos → 4 filas en trades, 1 posición en Binance → TP parcial/SL/trailing puede cerrar de más, dejar DB inconsistente o duplicar PnL.

Corrección: tracking de qty asignada por trade; al cerrar usar `min(qty_trade, disponible - ya_reservado)`; o una sola fila por símbolo con escalones internos.

AUD-S18 · Cambio de lado sin cancelar límites opuestas (ALTA)

Ubicación: `refrescar_grid()` cuando `abiertos == 0`

Problema: si RSI cruza 50, side cambia pero límites BUY previas (u otro side) en DB/Binance no se cancelan.

Corrección: al determinar side, cancelar todas las límites cuyo side `!= side_actual`.

AUD-S19 · Race condition en detección FILLED (ALTA)

Ubicación: `revisar_llenados()`

Problema: entre leer `status == FILLED` y `DELETE FROM limites`, otra pasada del hilo (5s) puede registrar el mismo fill dos veces → trades duplicados, doble SL.

Corrección: transacción atómica; flag processed en DB; o `INSERT OR IGNORE` con PK en (`symbol`, `order_id`).

AUD-S20 · Índice de escalón inconsistente (MEDIA)

- Tabla `limites`: escalón 0..3 (0-based).
- Tabla `trades`: guarda `escalon+1` (1-based) línea 762.

Impacto: confusión operativa en logs/Telegram y posibles off-by-one si se reutiliza lógica.

Corrección: unificar convención y documentar.

AUD-S21 · SL real fallido → solo aviso (MEDIA — operativo)

Ubicación: líneas 755–773

Problema: si `poner_stop_real` falla tras reintento, el trade queda con SL “virtual” del monitor. Si el bot cae, **sin protección en exchange**.

Corrección: reintentos agresivos, alerta Telegram crítica, opcionalmente cerrar a mercado si no se puede poner SL.

AUD-S22 · /estado crashea hilo Telegram (ALTA)

Ubicación: línea 942 — `f"Balance libre: {bal:.2f}"` si `bal` is None.

Corrección: `f"{bal:.2f}" if bal is not None else "N/D"`.

6. Base de datos SQLite

AUD-S23 · Sin modo WAL ni migraciones (MEDIA)

Problema: escrituras concurrentes desde 5 hilos; `timeout=30` ayuda pero WAL mejora lecturas paralelas.

Corrección: `PRAGMA journal_mode=WAL` en `init_db()`.

AUD-S24 · Sin backup automático (MEDIA)

Problema: corrupción o borrado → el limpiador cancela órdenes “no rastreadas” (líneas 1028–1031).

Corrección: backup periódico de .db; snapshot antes de /aplicar_lista.

AUD-S25 · Locks incompletos (MEDIA)

Problema: revisar_llenados lee estado de orden fuera de db_lock; ventana de carrera con refrescar_grid.

Corrección: sección crítica unificada o cola single-thread para mutaciones de órdenes.

7. Hilos y ciclo de vida

AUD-S28 · Sin apagado graceful (BAJA)

Problema: hilos daemon; kill -9 deja límites abiertas (mitigado parcialmente por limpiador).

Corrección: handler SIGTERM → /pausa + flush DB.

AUD-S29 · modo_pausa sin lock (BAJA)

Problema: en teoría lectura/escritura concurrente; en CPython bool suele ser seguro pero no garantizado.

Corrección: threading.Event o lock.

8. Observabilidad

AUD-S30 · Solo logging a consola (MEDIA)

Corrección: RotatingFileHandler, correlación con trade_id, nivel DEBUG opcional por env.

AUD-S31 · Errores silenciados (MEDIA)

Patrón repetido: except Exception: pass en cancelaciones.

Corrección: al menos log.debug con contexto.

9. Integración con auto_auditor

AUD-S32 · Contrato de integración OK (INFO)

correr_auditoria() llama correctamente a auto_auditor.auditar_universo(). Requiere módulo renombrado (AUD-S01).

AUD-S33 · Apalancamiento 25 hardcodeado en propuesta (ALTA)

Ubicación: línea 1126 — "apal": 25 al armar propuesta.

Problema: al aplicar lista, símbolos con max leverage menor fallarán (relacionado AUD-S07).

10. Matriz de priorización de fixes

ID	Fix	Esfuerzo
----	-----	-----

ID	Fix	Esfuerzo
AUD-S01	Renombrar/import auto_auditor	5 min
AUD-S04	Env vars + rotar keys	30 min
AUD-S05	Filtro chat Telegram	10 min
AUD-S07	Leverage máximo por símbolo	1–2 h
AUD-S15	RSI unificado Wilder	1 h
AUD-S17	Fix multi-escalón qty	2–4 h
AUD-S18	Cancel límites lado opuesto	30 min
AUD-S19	Idempotencia FILLED	1 h
AUD-S08	recvWindow + reintentos	1 h
AUD-S22	Fix /estado None	5 min

11. Checklist pre-producción

- `auto_auditor.py` en mismo directorio
- `pip install -r requirements.txt`
- Secretos en entorno, keys rotadas
- Cuenta en one-way mode
- Reloj NTP sincronizado
- DB path absoluto
- Filtro Telegram chat_id
- Leverage por símbolo validado
- Prueba en testnet o 1 símbolo mínimo antes de 17

Fin del informe SNIPER RSI v2